

Replication

1. What is Replication?

Replication means creating multiple copies of the same data and storing them on different database nodes/servers.

None

Original Data → Copy 1 → Copy 2 → Copy 3

Each copy is called a **replica**.

Used in distributed databases to improve:

- Availability
- Fault tolerance
- Read scalability
- Disaster recovery
- Geographic performance

2. Why Replication Is Needed

Single database server has limits:

None

CPU limit

RAM limit

Disk limit

Network bandwidth limit

Single point of failure

So we distribute copies across multiple machines.

3. Main Benefits of Replication

A. Fault Tolerance

If one node crashes:

None

Replica takes over

System continues working.

B. Higher Read Throughput

Many users can read from multiple replicas.

Instead of:

None

1 DB handling all reads

Use:

None

10 replicas handling reads

C. Lower Latency

Place replicas near users.

Example:

- India users read from Mumbai region
- Europe users read from Frankfurt region

D. Backup / Disaster Recovery

If data center fails, another region replica survives.

4. Replication Does NOT Automatically Scale Writes

Reads are easy to scale using replicas.

Writes are harder because:

None

Every replica must receive updates

Need synchronization.

5. Common Replication Architectures

A. Primary-Replica (Leader-Follower)

One primary handles writes.

Replicas copy changes.

None

App → Primary (write)

App → Replica (read)

Examples:

- MySQL
- PostgreSQL

B. Multi-Primary

Multiple nodes accept writes.

Useful for multi-region systems.

But conflict resolution needed.

C. Peer-to-Peer

All nodes equal.

Example:

- Apache Cassandra

6. Synchronous vs Asynchronous Replication

Synchronous

Write succeeds only after replicas confirm.

✓ Strong consistency

✗ Higher latency

Asynchronous

Primary confirms immediately, replicas update later.

✓ Fast writes

✗ Replica lag possible

7. Example

User places order.

None

Primary writes order

Replica copies after 100 ms

If read hits lagging replica immediately:

User may not see order yet.

Called:

None

Replication Lag

8. Replication vs Partitioning vs Sharding

Concept	Meaning
Replication	Same data copied multiple times

Partitioning	Split data into subsets
Sharding	Partition subsets across multiple nodes

Example:

None

Users A-M on Node1

Users N-Z on Node2

That is sharding.

Each shard may also have replicas.

9. HLD Design Decisions

When designing replication discuss:

- Number of replicas
- Read/write ratio
- Cross-region replicas
- Failover strategy
- Sync vs async
- Consistency requirement
- Recovery time objective (RTO)
- Recovery point objective (RPO)

10. Interview Example

Q: How to scale read-heavy system?

Use one write primary and multiple read replicas. Route SELECT queries to replicas and writes to primary.

11. Trade-offs

Advantages

High availability

Better reads

Lower latency globally

Backup protection

Disadvantages

Write complexity

Replica lag

Failover complexity

More infra cost

12. Real World Examples

- Amazon regional replicas
- Netflix multi-region databases
- Meta Platforms globally replicated systems

13. Short Revision Notes

None

Replication =

copy same data to many nodes

Used for:

HA

read scaling

backup

low latency

Write scaling harder than read scaling

14. One-Line Summary

Replication improves availability, read scalability, and resilience by storing multiple synchronized copies of data across different database nodes.